**TED 03 – Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)**

| | | | | |
|---|---|---|---|---|
| **CURSO** | Análise e Desenvolvimento de Sistemas | **PERÍODO** | 2026.2 | |
| **UNI. CURRICULAR** | Algoritmos e Programação Avançada | | | |
| **ACADÊMICO(S)** | Marcos Vinícius Sousa dos Santos<br>Esley Robson Soares Silva Filho | | | |
| **PROFESSOR** | Alex Rogaleski Marques | | | |

## 2. Introdução e objetivos

Este relatório apresenta o desenvolvimento do TED 03, cuja proposta é construir, em Python, um módulo de gerenciamento de dados persistidos em arquivos externos nos formatos CSV e JSON. A aplicação realiza a leitura, a escrita, a conversão e a atualização desses dados, tratando falhas de Entrada/Saída (E/S) e criando uma cópia de segurança antes de qualquer alteração em um arquivo já existente, de modo a preservar a integridade das informações.

Os objetivos do trabalho foram:

- implementar a leitura de arquivos CSV e JSON e a conversão entre os dois formatos, sem perda de registros;
- utilizar o gerenciador de contexto `with open(...)`, com codificação UTF-8 explícita, em todas as aberturas de arquivo;
- tratar `FileNotFoundError`, `PermissionError`, `UnicodeDecodeError` e outras falhas, exibindo mensagens claras ao usuário;
- criar backup automático antes de qualquer gravação em arquivo existente, informando o usuário;
- organizar o código em módulos, separando leitura, escrita, conversão, backup e tratamento de erros.

## 3. Descrição da solução

### 3.1 Organização do código

A solução utiliza apenas a biblioteca padrão do Python e está dividida em módulos, cada um com uma responsabilidade:

- `main.py`: menu interativo no terminal (converter CSV para JSON, converter JSON para CSV, mostrar registros e adicionar registro);
- `leitura.py`: leitura de arquivos CSV e JSON, retornando os dados como lista de dicionários;
- `escrita.py`: gravação de arquivos CSV e JSON, sempre precedida do backup;
- `conversao.py`: conversões CSV para JSON e JSON para CSV;
- `backup.py`: criação do backup automático antes de alterar um arquivo existente;

- `registros.py`: inclusão de novos registros em arquivos já existentes;
- `erros.py`: tradução das exceções em mensagens claras para o usuário;
- `testes.py`: testes automatizados (unittest) das conversões, do backup e das situações de erro.

A pasta `dados/` reúne os arquivos de teste: `alunos.csv` e `alunos.json` (arquivos válidos), `alunos_latin1.csv` (CSV salvo fora do UTF-8, para provocar erro de codificação) e `invalido.json` (JSON com erro de sintaxe).

### 3.2 Manipulação e persistência dos dados

Na leitura, cada linha do CSV é convertida em um dicionário cujas chaves são os nomes das colunas do cabeçalho (`csv.DictReader`), e o JSON é carregado com `json.load`. Dessa forma, os dois formatos passam a ser representados internamente do mesmo modo: uma lista de registros. A gravação em JSON usa `json.dump` (com `ensure_ascii=False`, para manter os acentos legíveis) e a gravação em CSV usa `csv.DictWriter`.

Para evitar perda de informação na conversão, adotaram-se as seguintes decisões: (i) os valores lidos do CSV permanecem como texto, preservando, por exemplo, os zeros à esquerda de uma matrícula como `00123`; (ii) na conversão de JSON para CSV, as colunas são a união dos campos de todos os registros, de forma que um campo presente em apenas um registro não é descartado (nos demais, a célula fica vazia); (iii) um JSON que contenha um único objeto é tratado como uma lista com um registro, e um JSON que não seja uma lista de objetos gera uma mensagem de erro.

### 3.3 Fluxo de uma gravação com backup

Toda gravação segue a mesma sequência: o programa lê a origem; chama a função de backup para o arquivo de destino; se o backup foi concluído (ou se o destino ainda não existe), grava o novo conteúdo; se o backup falhar, a gravação é cancelada e o arquivo original permanece intacto. Os backups são salvos na pasta `backups/`, criada ao lado do arquivo, com data e hora no nome (por exemplo, `20261031_190730_alunos_convertido.json.bak`).

## 4. Implementação

Esta seção apresenta os trechos mais relevantes do código-fonte, extraídos dos arquivos do projeto.

### 4.1 Uso do gerenciador de contexto with open(...)

Todas as aberturas de arquivo utilizam `with open(...)` com `encoding="utf-8"`, o que garante o fechamento automático do arquivo ao fim do bloco, inclusive quando ocorre uma exceção. No CSV, o parâmetro `newline=""` é o recomendado pelo módulo `csv` para o tratamento correto das quebras de linha. Trecho do arquivo `leitura.py`:

```
def ler_csv(caminho):
    with open(caminho, "r", encoding="utf-8", newline="") as arquivo:
        leitor = csv.DictReader(arquivo)
        dados = list(leitor)
    return dados

def ler_json(caminho):
    with open(caminho, "r", encoding="utf-8") as arquivo:
        dados = json.load(arquivo)
    return dados
```

### 4.2 Leitura, escrita e conversão entre CSV e JSON

A conversão lê os dados da origem para a memória e os entrega à função de gravação do formato de destino. Em `json_para_csv`, a validação garante que o JSON seja uma lista de objetos antes da gravação. Trecho do arquivo `conversao.py`:

```
def csv_para_json(origem, destino):
    dados = ler_csv(origem)
    if salvar_json(destino, dados):
        print(f"{len(dados)} registro(s) convertido(s) de '{origem}' para '{destino}'.")

def json_para_csv(origem, destino):
    dados = ler_json(origem)

    # se vier um objeto só, vira uma lista com um registro
    if isinstance(dados, dict):
        dados = [dados]

    if not isinstance(dados, list) or not all(isinstance(item, dict) for item in dados):
        raise ValueError("o JSON precisa ser uma lista de objetos (registros)")

    if salvar_csv(destino, dados):
        print(f"{len(dados)} registro(s) convertido(s) de '{origem}' para '{destino}'.")
```

A função `salvar_csv`, do arquivo `escrita.py`, reúne os nomes de todas as colunas existentes nos registros e só então grava o arquivo, depois de chamar o backup:

```
def salvar_csv(caminho, dados):
    if len(dados) == 0:
        raise ValueError("não há registros para gravar no CSV")

    # junta as colunas de todos os registros, porque nem todos
    # têm os mesmos campos (isso acontece muito em JSON)
    colunas = []
    for registro in dados:
        for campo in registro:
            if campo not in colunas:
                colunas.append(campo)

    if not fazer_backup(caminho):
        return False

    with open(caminho, "w", encoding="utf-8", newline="") as arquivo:
        escritor = csv.DictWriter(arquivo, fieldnames=colunas)
        escritor.writeheader()
        escritor.writerows(dados)
    return True
```

### 4.3 Tratamento de exceções

Cada opção do menu executa a operação dentro de um bloco `try/except`, de modo que nenhuma falha encerre o programa de forma inesperada. A função `mensagem_erro`, do arquivo `erros.py`, converte a exceção capturada em uma mensagem clara. A ordem das verificações é importante, pois `JSONDecodeError` e `UnicodeDecodeError` são tipos de `ValueError`.

```
        elif opcao == "1":
            origem = pedir("CSV de origem", "dados/alunos.csv")
            destino = pedir("JSON de destino", "dados/alunos_convertido.json")
            try:
                csv_para_json(origem, destino)
            except Exception as erro:
                print(mensagem_erro(erro, origem))
```

```
def mensagem_erro(erro, caminho=""):
    # nos erros de arquivo o Python já diz qual arquivo deu problema
    if isinstance(erro, OSError) and erro.filename:
        caminho = erro.filename

    # a ordem importa: JSONDecodeError e UnicodeDecodeError são tipos de ValueError
    if isinstance(erro, FileNotFoundError):
        return f"Erro: o arquivo '{caminho}' não foi encontrado."
    if isinstance(erro, PermissionError):
        return f"Erro: sem permissão para acessar ou modificar o arquivo '{caminho}'."
    if isinstance(erro, UnicodeDecodeError):
        return f"Erro: o arquivo '{caminho}' não está em UTF-8, não foi possível ler os caracteres."
    if isinstance(erro, json.JSONDecodeError):
        return f"Erro: '{caminho}' não é um JSON válido ({erro.msg}, linha {erro.lineno})."
    if isinstance(erro, csv.Error):
        return f"Erro: o CSV '{caminho}' está mal formatado ({erro})."
    if isinstance(erro, ValueError):
        return f"Erro: {erro}."
    if isinstance(erro, OSError):
        return f"Erro de leitura/escrita no arquivo '{caminho}': {erro.strerror}."
    return f"Erro inesperado: {erro}"
```

### 4.4 Backup automático antes da escrita

A função `fazer_backup` copia o arquivo existente para a pasta `backups/` antes de qualquer alteração. A cópia é feita em modo binário (`rb`/`wb`), que reproduz os bytes originais sem depender de codificação. A função retorna `True` quando a gravação pode continuar e `False` quando o backup falhou; nesse caso, informa o problema ao usuário e a gravação é cancelada. Trecho do arquivo `backup.py`:

```
def fazer_backup(caminho):
    # retorna True se pode continuar com a gravação e False se o backup deu errado
    if not os.path.exists(caminho):
        # arquivo novo, não tem nada pra copiar
        return True

    pasta = os.path.join(os.path.dirname(caminho), "backups")
    nome = os.path.basename(caminho)
    data = datetime.now().strftime("%Y%m%d_%H%M%S")
    destino = os.path.join(pasta, f"{data}_{nome}.bak")

    try:
        os.makedirs(pasta, exist_ok=True)
        # copia em modo binário pra não ter problema com encoding
        with open(caminho, "rb") as origem:
            with open(destino, "wb") as copia:
                shutil.copyfileobj(origem, copia)
    except OSError as erro:
        print(f"Não foi possível criar o backup de '{caminho}': {erro}")
        print("A gravação foi cancelada pra não perder os dados originais.")
        return False

    print(f"Backup criado em: {destino}")
    return True
```

A chamada ocorre dentro das funções de gravação, sempre antes da abertura do arquivo para escrita (trecho de `escrita.py`):

```
def salvar_json(caminho, dados):
    # o backup sempre vem antes de mexer no arquivo
    if not fazer_backup(caminho):
        return False

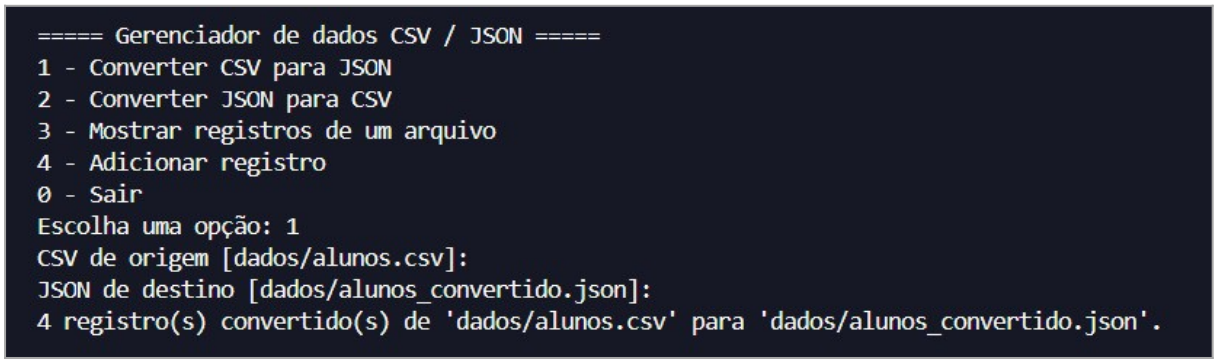
    with open(caminho, "w", encoding="utf-8") as arquivo:
        json.dump(dados, arquivo, ensure_ascii=False, indent=4)
    return True
```

## 5. Testes e evidências de execução

Os testes foram executados em terminal, com o comando `python main.py` a partir da pasta do projeto, utilizando os arquivos de exemplo da pasta `dados/`. Para cada cenário são apresentados a captura de tela, o contexto do teste e o resultado observado.

### 5.1 Leitura de um arquivo CSV e conversão para JSON

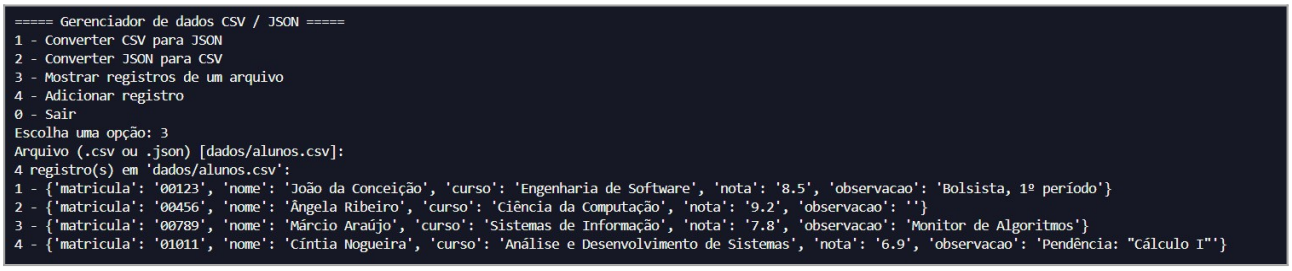
**Figura 1: Conversão de CSV para JSON (opção 1 do menu)**




Fonte: elaborado pelos autores (2026).

**Contexto:** foi escolhida a opção 1, mantendo os caminhos padrão `dados/alunos.csv` (origem) e `dados/alunos_convertido.json` (destino). **Resultado:** o programa leu os 4 registros do CSV e gravou o arquivo JSON, exibindo a confirmação da conversão. Como o arquivo de destino ainda não existia, não havia dados anteriores a preservar e nenhuma mensagem de backup foi exibida.

**Figura 2: Leitura e listagem dos registros do CSV de origem (opção 3 do menu)**



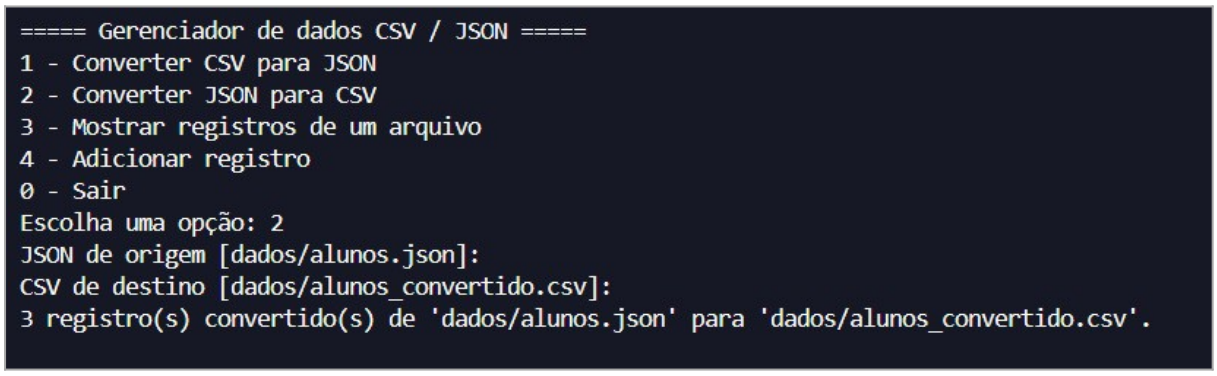

Fonte: elaborado pelos autores (2026).

**Contexto:** foi escolhida a opção 3 para exibir o conteúdo de `dados/alunos.csv`. **Resultado:** os 4 registros foram lidos corretamente. Observa-se que os acentos foram preservados (João, Ângela, Márcio, Cíntia), que a matrícula manteve os zeros à esquerda (`'00123'`), que a vírgula dentro de um campo (`Bolsista, 1º período`) e as aspas internas (`"Cálculo I"`) foram interpretadas corretamente e que o campo vazio foi lido como texto vazio. Como a imagem é larga, a saída é transcrita abaixo para facilitar a leitura (apenas com quebras de linha adicionadas):

```
4 registro(s) em 'dados/alunos.csv':
1 - {'matricula': '00123', 'nome': 'João da Conceição', 'curso': 'Engenharia de
    Software', 'nota': '8.5', 'observacao': 'Bolsista, 1º período'}
2 - {'matricula': '00456', 'nome': 'Ângela Ribeiro', 'curso': 'Ciência da
    Computação', 'nota': '9.2', 'observacao': ''}
3 - {'matricula': '00789', 'nome': 'Márcio Araújo', 'curso': 'Sistemas de
    Informação', 'nota': '7.8', 'observacao': 'Monitor de Algoritmos'}
4 - {'matricula': '01011', 'nome': 'Cíntia Nogueira', 'curso': 'Análise e
    Desenvolvimento de Sistemas', 'nota': '6.9', 'observacao': 'Pendência: "Cálculo
    I"'}
```

### 5.2 Leitura de um arquivo JSON e conversão para CSV

**Figura 3: Conversão de JSON para CSV (opção 2 do menu)**

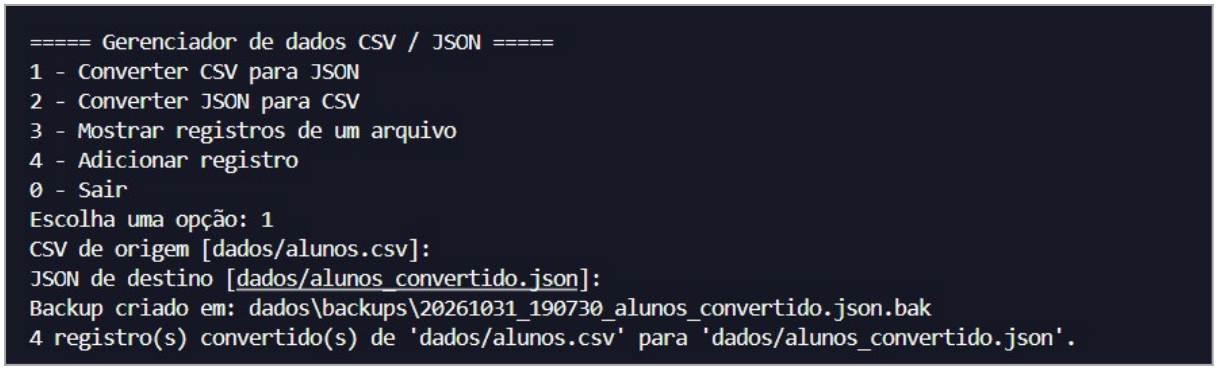



Fonte: elaborado pelos autores (2026).

**Contexto:** foi escolhida a opção 2, com os caminhos padrão `dados/alunos.json` (origem) e `dados/alunos_convertido.csv` (destino). **Resultado:** os 3 registros do JSON foram convertidos para CSV. O arquivo de teste possui um registro com um campo a mais (`observacao`); por isso o cabeçalho do CSV é formado pela união dos campos de todos os registros, sem descartar nenhum dado.

### 5.3 Operações de escrita e atualização com geração prévia de backup

**Figura 4: Backup criado antes de sobrescrever o JSON existente**




Fonte: elaborado pelos autores (2026).

**Contexto:** a opção 1 foi executada novamente com os mesmos caminhos. O arquivo `dados/alunos_convertido.json` já existia, pois havia sido criado no teste da seção 5.1. **Resultado:** antes de gravar, o programa criou e informou o backup `dados\backups\20261031_190730_alunos_convertido.json.bak` e só depois concluiu a conversão, garantindo que a versão anterior do arquivo não fosse perdida.

**Figura 5: Adição de registro (atualização) em arquivo existente, com backup**

```
===== Gerenciador de dados CSV / JSON =====
1 - Converter CSV para JSON
2 - Converter JSON para CSV
3 - Mostrar registros de um arquivo
4 - Adicionar registro
0 - Sair
Escolha uma opção: 4
Arquivo (.csv ou .json) [dados/alunos.csv]:
matricula: 20260001
nome: Teste
curso: Direito
nota: 8.0
observacao:
Backup criado em: dados\backups\20261031_191432_alunos.csv.bak
Registro adicionado.
```

Fonte: elaborado pelos autores (2026).

**Contexto:** foi escolhida a opção 4 sobre `dados/alunos.csv`, informando os valores de cada campo (matrícula `20260001`, nome `Teste`, curso `Direito`, nota `8.0` e observação em branco). **Resultado:** o programa criou o backup `dados\backups\20261031_191432_alunos.csv.bak`, informou o usuário e confirmou a inclusão com a mensagem “Registro adicionado.”, mostrando que a cópia de segurança é feita também nas operações de atualização.

### 5.4 Tratamento de situações de erro com arquivos

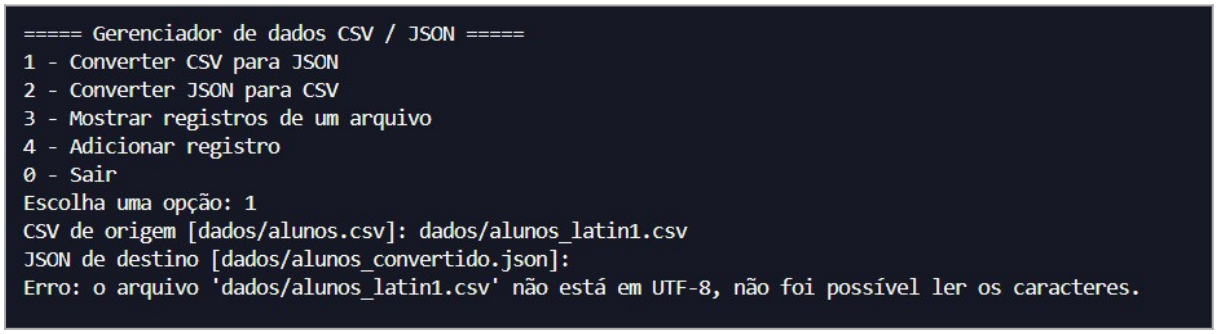
**Figura 6: Erro: arquivo inexistente (FileNotFoundError)**

```
===== Gerenciador de dados CSV / JSON =====
1 - Converter CSV para JSON
2 - Converter JSON para CSV
3 - Mostrar registros de um arquivo
4 - Adicionar registro
0 - Sair
Escolha uma opção: 1
CSV de origem [dados/alunos.csv]: dados/nao_existe.csv
JSON de destino [dados/alunos_convertido.json]:
Erro: o arquivo 'dados/nao_existe.csv' não foi encontrado.
```

Fonte: elaborado pelos autores (2026).

**Contexto:** na opção 1, foi informado como origem o arquivo `dados/nao_existe.csv`, que não existe. **Resultado:** o programa não foi encerrado e exibiu a mensagem “Erro: o arquivo 'dados/nao_existe.csv' não foi encontrado.”, indicando claramente o problema e o arquivo envolvido.

**Figura 7: Erro: arquivo fora da codificação UTF-8 (UnicodeDecodeError)**




Fonte: elaborado pelos autores (2026).

**Contexto:** na opção 1, foi informado o arquivo `dados/alunos_latin1.csv`, que foi salvo na codificação Latin-1 e contém acentos. **Resultado:** como a leitura é feita em UTF-8, ocorreu um `UnicodeDecodeError`, tratado pelo programa com a mensagem "não está em UTF-8, não foi possível ler os caracteres", sem interromper a execução.

**Figura 8: Erro: JSON malformado (JSONDecodeError)**




Fonte: elaborado pelos autores (2026).

**Contexto:** na opção 2, foi informado o arquivo `dados/invalido.json`, que possui uma vírgula sobrando após o último campo do objeto. **Resultado:** o programa exibiu a mensagem "não é um JSON válido" acompanhada do detalhe fornecido pela biblioteca `json` do Python (vírgula final ilegal, na linha 2). Esse detalhe pode variar conforme a versão do Python utilizada.

## 6. Conclusão

Os resultados obtidos mostram que os objetivos do trabalho foram alcançados. A aplicação lê e grava arquivos CSV e JSON, converte os dados nos dois sentidos preservando os registros e os valores dos campos, utiliza `with open(...)` com UTF-8 em todas as aberturas de arquivo, trata os erros de arquivo com mensagens claras e cria um backup automático antes de qualquer gravação em arquivo existente. Também foi implementada a inclusão de novos registros, que reaproveita os mesmos mecanismos de leitura, escrita e backup. O erro de permissão (`PermissionError`) está previsto no tratamento de exceções e é verificado por teste automatizado da mensagem, embora não conste entre as capturas de tela deste relatório.

Entre as dificuldades encontradas, destacam-se: (i) a execução do programa fora da pasta do projeto, que faz os caminhos relativos (`dados/...`) não serem localizados e resulta no erro de arquivo não encontrado; (ii) a preservação dos zeros à esquerda dos campos numéricos, resolvida mantendo os valores do CSV como texto; (iii) a conversão de JSON para CSV quando os registros têm campos diferentes, resolvida com a união das colunas; (iv) a garantia de que o backup ocorra antes da escrita e de que a gravação seja cancelada se ele falhar; e (v) o fato de a mensagem detalhada do erro de JSON variar conforme a versão do Python.

Como limitações, a leitura de CSV considera apenas a vírgula como separador de campos, e valores aninhados em um JSON (listas ou objetos dentro de um registro) não recebem tratamento especial na conversão para CSV. Esses pontos podem ser aprimorados em versões futuras.

## 7. Referências

PYTHON SOFTWARE FOUNDATION. **csv: CSV File Reading and Writing**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/csv.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **json: JSON encoder and decoder**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/json.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Built-in Functions: open()**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/functions.html#open. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Built-in Exceptions**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/exceptions.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **shutil: High-level file operations**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/shutil.html. Acesso em: 31 out. 2026.

PYTHON SOFTWARE FOUNDATION. **unittest: Unit testing framework**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/unittest.html. Acesso em: 31 out. 2026.

MARQUES, Alex Rogaleski. **Materiais e aulas da disciplina de Algoritmos e Programação Avançada**. Centro Universitário Unibalsas. 2026.